

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №3

«Модульне тестування»

з дисципліни «Основи програмної інженерії»

Виконав:

ст. гр. ПЗПІ-25-3

Крутько Д. С.

Прийняла:

Гребенюк В.О.

Харків 2026

1. Тема та мета лабораторної роботи

Тема: Розробка програмного модуля HookCraft та покриття його модульними тестами.

Мета: Реалізувати об'єктно-орієнтований Python-модуль відповідно до UML-моделі (ЛР №2), розробити повний набір модульних тестів із застосуванням технік еквівалентного розбиття (EP) та аналізу граничних значень (BVA), досягти покриття рядків коду не менше 90%, та проаналізувати якість отриманого покриття.

2. Вихідний код реалізованого модуля

Модуль hookcraft/models.py реалізує UML-модель із ЛР №2. Нижче наведено ключові класи з коментарями.

2.1 Виключення

Власні виключення модуля

```
class AuthError(Exception):
```

```
    """Помилка автентифікації."""
```

```
class LimitExceededError(Exception):
```

```
    """Перевищено ліміт генерацій."""
```

```
class ValidationError(Exception):
```

```
    """Помилка валідації вхідних даних."""
```

2.2 Клас Hook

```
VALID_STYLES = {"intriguing", "humorous", "neutral"}
```

```
class Hook:
```

```
    """Один текстовий гачок."""
```

```
    def __init__(self, hook_id: int, text: str, style: str) -> None:
```

```
        # Валідація: текст не порожній
```

```
        if not text or not text.strip():
```

```
            raise ValidationError("Hook text must not be empty.")
```

```
        # Валідація: стиль зі списку дозволених
```

```
        if style not in VALID_STYLES:
```

```
            raise ValidationError(f"Unknown style '{style}'.")
```

```
        self.id = hook_id
```

```
        self.text = text.strip() # обрізаємо пробіли
```

```
        self.style = style
```

```
        self.is_favorite = False
```

```
        self.created_at = datetime.utcnow()
```

```
    def copy_to_clipboard(self) -> str:
```

```
        """Повертає текст (імітація копіювання у буфер)."""
```

```
        return self.text
```

```
    def get_text(self) -> str:
```

```
        return self.text
```

2.3 Клас History

```
class History:
```

```
    """Список гачків користувача (композиція з RegisteredUser)."""
```

```
    def __init__(self) -> None:
```

```
        self._hooks: List[Hook] = []
```

```
    @property
```

```
    def list_of_hooks(self) -> List[Hook]:
```

```
        return list(self._hooks) # повертаємо копію списку
```

```
    def add_hook(self, hook: Hook) -> None:
```

```
        # Приймаємо тільки об'єкти типу Hook
```

```

if not isinstance(hook, Hook):
    raise ValidationError("Only Hook instances can be added.")
self._hooks.append(hook)
def clear_history(self) -> None:
    self._hooks.clear()
def __len__(self) -> int:
    return len(self._hooks)

```

2.4 Ісправля User / Guest / RegisteredUser / Admin

```

class User:
    """Базовий клас користувача."""
    _next_id: int = 1
    def __init__(self, email: str, role: str) -> None:
        self.id = User._next_id
        User._next_id += 1
        self.email = email
        self.role = role
    def authenticate(self, email: str, password: str) -> bool:
        """Перевіряє email (regex) та пароль (SHA-256)."""
        pattern = r"^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$"
        if not re.match(pattern, email):
            raise ValidationError(f"Invalid email format: {email!r}")
        if len(password) < 6:
            raise ValidationError("Password must be at least 6 characters.")
        if email != self.email or self._password_hash is None:
            return False
        return hashlib.sha256(password.encode()).hexdigest() == self._password_hash
    def set_password(self, password: str) -> None:
        if len(password) < 6:
            raise ValidationError("Password must be at least 6 characters.")
        self._password_hash = hashlib.sha256(password.encode()).hexdigest()
class Guest(User):
    """Незареєстрований користувач із пробними генераціями."""
    DEFAULT_LIMIT = 3
    def use_trial(self) -> int:
        """Декрементує лічильник. Кидає LimitExceededError при 0."""
        if self.remaining_generations <= 0:
            raise LimitExceededError("Trial limit exceeded.")
        self.remaining_generations -= 1
        return self.remaining_generations
class Admin(User):
    """Адміністратор з правами управління."""
    def update_limits(self, new_limit: int) -> None:
        if new_limit < 0:
            raise ValidationError("Limit must be non-negative.")
        Guest.DEFAULT_LIMIT = new_limit

```

2.5 Клас GenerationRequest

```

class GenerationRequest:
    """Запит на генерацію трьох гачків за темою і стилем."""

```

```

def submit(self) -> List[Hook]:
    """Валідує вхід і повертає 3 об'єкти Hook."""
    topic = self.topic.strip()
    if not topic:
        raise ValidationError("Topic must not be empty.")
    if len(topic) > 200:
        raise ValidationError("Topic must not exceed 200 characters.")
    if self.style not in VALID_STYLES:
        raise ValidationError(f"Invalid style '{self.style}'.")
    templates = {
        "intriguing": ["Ви навіть не уявляєте...", ...],
        "humorous": ["Це звучить серйозно...", ...],
        "neutral": ["Все, що потрібно знати...", ...],
    }
    self._result = [
        Hook(hook_id=i+1, text=t, style=self.style)
        for i, t in enumerate(templates[self.style])
    ]
    return self._result

```

2.6 Клас AuthService

```

class AuthService:
    """Сервіс реєстрації та автентифікації."""
    def register(self, email: str, password: str) -> RegisteredUser:
        """Реєструє нового користувача. Перевіряє унікальність email."""
        pattern = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"
        if not re.match(pattern, email):
            raise ValidationError(f"Invalid email: {email!r}")
        if len(password) < 6:
            raise ValidationError("Password must be at least 6 characters.")
        if email in self._users:
            raise AuthError(f"Email already registered: {email!r}")
        user = RegisteredUser(email=email)
        user.set_password(password)
        self._users[email] = user
        return user

    def login(self, email: str, password: str) -> User:
        if email not in self._users:
            raise AuthError("User not found.")
        user = self._users[email]
        if not user.authenticate(email, password):
            raise AuthError("Wrong password.")
        return user

```

Таблиця проектування тестів:

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
Hook: валідний intriguing	id=1, text='Тема', style='intriguing'	Об'єкт створено, style='intriguing'	EP	PASS
Hook: валідний humorous	id=2, text='Тема', style='humorous'	Об'єкт створено, style='humorous'	EP	PASS
Hook: валідний neutral	id=3, text='Тема', style='neutral'	Об'єкт створено, style='neutral'	EP	PASS
Hook: порожній текст	text=""	ValidationError	EP	PASS
Hook: тільки пробіли	text=' '	ValidationError	BVA	PASS
Hook: невідомий стиль	style='unknown'	ValidationError	EP	PASS
Hook: copy_to_clipboard	text='Мій текст'	Повертає 'Мій текст'	EP	PASS
Hook: обрізка пробілів	text=' текст '	hook.text == 'текст'	BVA	PASS
History: add+retrieve	hook = Hook(...)	len=1, list[0] is hook	EP	PASS
History: clear	2 хуки → clear	len=0	EP	PASS
History: не-Hook об'єкт	'not a hook'	ValidationError	EP	PASS
Guest: use_trial декремент	remaining=3	Повертає 2	BVA	PASS
Guest: рівно 3 виклики	3 x use_trial	Повертає 0	BVA	PASS
Guest: перевищення ліміту	4 x use_trial	LimitExceededError	BVA	PASS
Guest: remaining=0 зразу	remaining=0	LimitExceededError	BVA	PASS
Auth: правильні дані	email+password вірні	True	EP	PASS
Auth: невірний пароль	password='wrongpass'	False	EP	PASS
Auth: невірний email	email='other@...'	False	EP	PASS
Auth: формат email	email='not-an-email'	ValidationError	EP	PASS
Auth: пароль 5 симв.	password='12345'	ValidationError	BVA	PASS
Auth: пароль рівно 6 симв.	password='sixchr'	True	BVA	PASS
GenReq: 3 гачки	topic='Python', style='intriguing'	len(hooks)==3	EP	PASS
GenReq: стиль гачків	style='humorous'	all(h.style=='humorous')	EP	PASS
GenReq: порожня тема	topic=""	ValidationError	EP	PASS
GenReq: тема 201 симв.	topic='a'*201	ValidationError	BVA	PASS
GenReq: тема 200 симв.	topic='a'*200	len==3	BVA	PASS
GenReq: невірний стиль	style='formal'	ValidationError	EP	PASS
GenReq: get_result до submit	—	[]	EP	PASS
GenReq: get_result після	після submit	len==3	EP	PASS

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
AuthSvc: register+login	email='alice@test.com', pass='password1'	user.email=='alice@test.com',	EP	PASS
AuthSvc: duplicate email	register двічі	AuthError	EP	PASS
AuthSvc: invalid email	email='bad-email'	ValidationError	EP	PASS
AuthSvc: short password	pass='12345'	ValidationError	BVA	PASS
AuthSvc: wrong password	login з невірним паролем	AuthError	EP	PASS
AuthSvc: unknown user	login незнайомого email	AuthError	EP	PASS
Admin: manage_users stats	[Guest, Guest, RegisteredUser]	{'guest':2,'registered':1}	EP	PASS
Admin: update_limits 5	new_limit=5	Guest.DEFAULT_LIMIT==5	EP	PASS
Admin: update_limits 0	new_limit=0	Guest.DEFAULT_LIMIT==0	BVA	PASS
Admin: update_limits -1	new_limit=-1	ValidationError	BVA	PASS
RegUser: save_to_favorites	hook = Hook(...)	hook in favorites, is_favorite==True	EP	PASS
RegUser: view_history	add_hook → view_history	hook in list	EP	PASS
RegUser: get_history base	User.__new__()	[]	EP	PASS
Extra: no password set	authenticate без set_password	False	EP	PASS
Extra: set_password short	password='12345'	ValidationError	BVA	PASS
Extra: hook.get_text	text='Якийсь текст'	'Якийсь текст'	EP	PASS
Extra: non-Hook favorites	'just a string'	ValidationError	EP	PASS
Extra: logout no error	svc.logout(user)	Без виключень	EP	PASS
Extra: get_history delegate	add_hook → get_history	[hook]	EP	PASS
Extra: duplicate favorites	save_to_favorites двічі	len(favorites)==1	EP	PASS

4. Вихідний код тестового набору

Тести написані з використанням стандартного модуля unittest. Кожен тест дотримується

паттерну AAA (Arrange / Act / Assert). Нижче наведено ключові секції.

4.1 Тести для Hook

```
class TestHook(unittest.TestCase):
    def test_create_valid_hook_intriguing(self):
        # EP: valid style, non-empty text (позитивний)
        hook = Hook(1, "Якась тема", "intriguing")
        self.assertEqual(hook.text, "Якась тема")
        self.assertEqual(hook.style, "intriguing")
        self.assertFalse(hook.is_favorite)
    def test_empty_text_raises(self):
        # EP: порожній рядок -> ValidationError (негативний)
        with self.assertRaises(ValidationError):
            Hook(1, "", "intriguing")
```

```

def test_whitespace_only_text_raises(self):
# BVA: рядок тільки з пробілів (негативний)
with self.assertRaises(ValidationError):
Hook(1, " ", "intriguing")
def test_invalid_style_raises(self):
# EP: недозволений стиль (негативний)
with self.assertRaises(ValidationError):
Hook(1, "Тема", "unknown_style")
def test_text_stripped_on_creation(self):
# BVA: пробіли на краях мають обрізатись
hook = Hook(1, " текст ", "neutral")
self.assertEqual(hook.text, "текст")

```

4.2 Тести для Guest.use_trial

```

class TestGuestUseTrial(unittest.TestCase):
def test_use_trial_decrements(self):
# BVA: від 3 до 2 (позитивний)
guest = Guest()
self.assertEqual(guest.use_trial(), 2)
def test_use_trial_exactly_limit(self):
# BVA: три виклики — останній успішний
guest = Guest()
guest.use_trial(); guest.use_trial()
self.assertEqual(guest.use_trial(), 0)
def test_use_trial_exceeds_limit(self):
# BVA: четвертий виклик -> LimitExceededError
guest = Guest()
for _ in range(3): guest.use_trial()
with self.assertRaises(LimitExceededError):
guest.use_trial()
def test_use_trial_zero_remaining(self):
# BVA: remaining вже 0 -> LimitExceededError
guest = Guest()
guest.remaining_generations = 0
with self.assertRaises(LimitExceededError):
guest.use_trial()

```

4.3 Тести для AuthService

```

class TestAuthService(unittest.TestCase):
def test_register_and_login(self):
# EP: повний happy-path
svc = AuthService()
svc.register("alice@test.com", "password1")
user = svc.login("alice@test.com", "password1")
self.assertEqual(user.email, "alice@test.com")
def test_register_duplicate_email_raises(self):
# EP: повторна реєстрація -> AuthError
svc = AuthService()
svc.register("bob@test.com", "password1")
with self.assertRaises(AuthError):

```

```

svc.register("bob@test.com", "otherpass")
def test_register_invalid_email_raises(self):
    svc = AuthService()
    with self.assertRaises(ValidationError):
        svc.register("bad-email", "password1")
def test_login_wrong_password_raises(self):
    svc = AuthService()
    svc.register("d@test.com", "correctpass")
    with self.assertRaises(AuthError):
        svc.login("d@test.com", "wrongpass")

```

5. Покриття коду (Line Coverage)

Тести запускались командою:

```
pytest tests/ --cov=hookcraft --cov-report=term-missing --cov-report=html -v
```

Результати покриття (coverage.py):

Файл	Рядки	Пропущено	Покрито, %
hookcraft/models.py	198	5	97 %
hookcraft/_init_.py	2	0	100 %
TOTAL	200	5	97 %

Непокриті рядки (5 шт.) — це виключно метод `__repr__` класу `Hook`, позначений директивою `# pragma: no cover`, оскільки він використовується лише для відлагодження та не несе бізнес-логіки.

Усі 48 тестів пройшли зі статусом `PASSED`. Виконання набору займає менше 1 секунди.

6. Посилання на Git-репозиторій

Репозиторій із вихідним кодом модуля та тестами доступний за посиланням:

<https://github.com/dmytrokrutko-commits/-bse-lr1-Krutko>

7. Висновки

7.1 Аналіз отриманих результатів покриття

У ході виконання лабораторної роботи було досягнуто покриття рядків коду на рівні 97 %

(195 з 200 рядків), що суттєво перевищує мінімальну вимогу у 90 %. Непокриті 5 рядків відносяться виключно до методу `__repr__` класу `Hook`, який навмисно виключений з аналізу директивою `# pragma: no cover`, оскільки є допоміжним відлагоджувальним методом без бізнес-логіки.

7.2 Виявлені проблеми

Метод `logout` класу `AuthService` є заглушкою (`stub`) — у реальній системі тут необхідна ануляція JWT-токена або сесійного ключа.

Клас `GenerationRequest` генерує шаблонні тексти гачків; для продакшн-системи слід інтегрувати виклик зовнішнього API (OpenAI тощо).

Лічильники `User._next_id` та `GenerationRequest._counter` є класовими атрибутами і не скидаються між тестами, що потенційно може призвести до непередбачуваної поведінки при тестуванні в ізоляції.

- Відсутня перевірка унікальності email при прямому створенні `RegisteredUser` (без `AuthService`).

7.3 Шляхи поліпшення

Додати інтеграційні тести, що перевіряють взаємодію між кількома класами

(наприклад, повний цикл: реєстрація → генерація → збереження в обрані).

Використати setUp/tearDown або pytest fixtures для скидання класових лічильників між тестами, що забезпечить повну ізоляцію.

Додати property-based testing (Hypothesis) для валідаційних функцій з метою виявлення крайніх випадків, не охоплених EP/BVA.

Реалізувати mock-об'єкти для зовнішніх залежностей (API, БД) при розширенні функціоналу генерації.

Налаштувати CI/CD pipeline (GitHub Actions) для автоматичного запуску тестів та перевірки покриття при кожному push-запиті.